

杂项

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	iii
第一章 Dancing Links	1
1.1 精确覆盖 (Dancing Links)	1
1.2 最少行重复覆盖 (Dancing Links)	5
第二章 离线算法	10
2.1 CDQ 分治	10
2.1.1 静态三维偏序计数	10
2.1.2 三维最长链、计数与方案	12
2.2 整体二分	17
2.2.1 整体二分：动态区间第 k 小	17
2.3 莫队算法	21
2.3.1 带修改莫队	21
2.3.1.1 带修改的莫队	21
2.3.2 树上莫队	24
2.3.2.1 树上莫队：点权与边权	24
2.3.3 回滚莫队	28
2.3.3.1 回滚莫队	28
2.3.4 莫队二次离线	32
2.3.4.1 莫队二次离线：异或位数配对	32
第三章 配套知识与验证范围	35
3.1 带修莫队：时间轴与区间状态	35
3.2 莫队二次离线：前缀事件与自身扣除	36
第一部分 跨类依赖	37
附录 A 数论	38
A.1 模算术简介	38
A.1.1 静态模整数	38
A.2 卢卡斯定理	40
A.2.1 Lucas：小素数模的大下标组合数	40
附录 B 组合数学	42
B.1 排列组合	42
B.1.1 阶乘组合数	42
附录 C 树状数组	44
C.1 树状数组第 k 小与组合依赖	44
附录 D 线段树	46
D.1 线段树基础	46
D.1.1 通用懒标记线段树与边界搜索	46

D.1.2	通用线段树与左右边界搜索	51
D.1.3	线段树：区间乘、加、赋值与模意义求和	56
附录 E	树上问题	59
E.1	树链剖分	59
E.1.1	重链剖分与 LCA	59

使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 Dancing Links

1.1 精确覆盖 (Dancing Links)

ExactCover(m,rows): 列 0..m-1, 各稀疏行内列号互异、无需排序; 行可重复或为空, 保留原 0-based 行号。solve() 返回 optional<vector<int>>, 无解为 nullopt, m=0 为有值的空解。只求任意首解, 不保证最少行数。成功/失败均恢复链与列计数, 可重复调用及复制; 实现字段不可外改, 移后源对象仅销毁或重新赋值。m 和行数 <INT_MAX, 1+m+ 非零数 ≤INT_MAX。构造 O(行数 +m+ 非零数), 空间 O(m+ 非零数), 递归深度 ≤m, 搜索最坏指数。

```
1 struct ExactCover
2 {
3     struct Node { int l, r, u, d, c, row; };
4     int m;
5     vector<Node> a;
6     vector<int> s, ans;
7
8     ExactCover(int columns, const vector<vector<int>>& rows) : m(columns)
9     {
10         assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
11         size_t z = size_t(m) + 1;
12         for (const auto &r : rows)
13         {
14             assert(r.size() <= size_t(INT_MAX) - z);
15             z += r.size();
16         }
17         a.reserve(z);
18         s.assign(m + 1, 0);
19         for (int c = 0; c <= m; c++)
20             a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, c, -1});
```

稀疏行建链与删除（接上页同一结构体）：

```

21     vector<int> seen(m, -1);
22     for (int i = 0; i < int(rows.size()); i++)
23     {
24         int h = -1;
25         for (int col : rows[i])
26         {
27             assert(0 <= col && col < m && seen[col] != i);
28             seen[col] = i;
29             int c = col + 1, x = a.size(), u = a[c].u;
30             a.push_back({x, x, u, c, c, i});
31             a[u].d = a[c].u = x;
32             s[c]++;
33             if (h == -1) h = x;
34             else
35             {
36                 int l = a[h].l;
37                 a[x].l = l; a[x].r = h;
38                 a[l].r = a[h].l = x;
39             }
40         }
41     }
42 }
43
44 void cover(int c)
45 {
46     a[a[c].l].r = a[c].r;
47     a[a[c].r].l = a[c].l;
48     for (int i = a[c].d; i != c; i = a[i].d)
49         for (int j = a[i].r; j != i; j = a[j].r)
50         {
51             a[a[j].u].d = a[j].d;
52             a[a[j].d].u = a[j].u;
53             s[a[j].c]--;
54         }
55 }

```

逆序恢复与搜索（接上页同一结构体）：

```

57 void uncover(int c)
58 {
59     for (int i = a[c].u; i != c; i = a[i].u)
60         for (int j = a[i].l; j != i; j = a[j].l)
61             {
62                 s[a[j].c]++;
63                 a[a[j].u].d = a[a[j].d].u = j;
64             }
65     a[a[c].l].r = a[a[c].r].l = c;
66 }
67
68 bool dfs()
69 {
70     int c = a[0].r;
71     if (!c) return true;
72     for (int j = a[c].r; j; j = a[j].r)
73         if (s[j] < s[c]) c = j;
74     cover(c);
75     for (int i = a[c].d; i != c; i = a[i].d)
76     {
77         ans.push_back(a[i].row);
78         for (int j = a[i].r; j != i; j = a[j].r) cover(a[j].c);
79         bool ok = dfs();
80         for (int j = a[i].l; j != i; j = a[j].l) uncover(a[j].c);
81         if (ok)
82             {
83                 uncover(c);
84                 return true;
85             }
86         ans.pop_back();
87     }
88     uncover(c);
89     return false;
90 }
91
92 optional<vector<int>>> solve()
93 {
94     ans.clear();
95     ans.reserve(m);
96     if (!dfs()) return nullopt;
97     return ans;
98 }
99 };

```

模板题使用：Luogu P4929 / 精确覆盖

题意：读入 $N \times M$ 的 01 矩阵， $N, M \leq 500$ 且非零数 ≤ 5000 ；每列恰被选择行覆盖一次。稀疏建模后输出原 1-based 行号，任一可行解均可；无解输出 No Solution!。

所需模板：ExactCover（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4929>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     vector<vector<int>> rows(n);
8     for (int i = 0; i < n; i++)
9         for (int j = 0, x; j < m; j++)
10            {
11                cin >> x;
12                if (x) rows[i].push_back(j);
13            }
14     ExactCover dlx(m, rows);
15     auto ans = dlx.solve();
16     if (!ans) cout << "No Solution!\n";
17     else
18     {
19         for (int i : *ans) cout << i + 1 << ' ';
20         cout << '\n';
21     }
22 }
```


1.2 最少行重复覆盖 (Dancing Links)

MinimumCover(m, rows), 列号 0..m-1, 每行内部互异, 允许空行/重复行。solve() 返回最少行数的一组原 0-based 行号; nullopt 无解, 有值空向量表示 m=0。每列至少一次, 允许重叠; 不含权值、覆盖次数或全解枚举。 $0 \leq m < \text{INT_MAX}$, 行数 $< \text{INT_MAX}$, $m+1 + \text{非零数} \leq \text{INT_MAX}$; 前提由调用方保证。构造 $O(\text{行数} + m + \text{非零数})$, 空间 $O(m + \text{非零数})$; 递归搜索最坏指数、深度 $\leq m$, 每状态 $O(m + \text{非零数})$ 。成功失败均恢复链, 可重复求解或复制; 重建用重新赋值。字段及辅助方法不作为可修改接口, 移动后源对象只销毁或赋值。下界选取不能被同一行共同覆盖的列, 至少还需同样多行; 按最少候选列搜索, 逆序恢复只改左右链。

```

1 struct MinimumCover
2 {
3     struct Node
4     {
5         int l, r, u, d, c, row;
6     };
7     int m, best = 0;
8     vector<Node> a;
9     vector<int> s, path, ans;
10    vector<unsigned char> mark;
11
12    MinimumCover(int columns, const vector<vector<int>> &rows) : m(columns)
13    {
14        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
15        size_t z = size_t(m) + 1;
16        for (const auto &r : rows)
17        {
18            assert(r.size() <= size_t(INT_MAX) - z);
19            z += r.size();
20        }
21        a.reserve(z);
22        s.assign(m + 1, 0);
23        mark.resize(m + 1);
24        path.reserve(m);
25        ans.reserve(m);
26        for (int c = 0; c <= m; c++)
27            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, c, -1});

```

稀疏建链与删列（接上页同一结构体）：

```

28     vector<int> seen(m, -1);
29     for (int i = 0; i < int(rows.size()); i++)
30     {
31         int h = -1;
32         for (int col : rows[i])
33         {
34             assert(0 <= col && col < m && seen[col] != i);
35             seen[col] = i;
36             int c = col + 1, x = a.size(), u = a[c].u;
37             a.push_back({x, x, u, c, c, i});
38             a[u].d = x;
39             a[c].u = x;
40             s[c]++;
41             if (h == -1) h = x;
42             else
43             {
44                 int l = a[h].l;
45                 a[x].l = l;
46                 a[x].r = h;
47                 a[l].r = x;
48                 a[h].l = x;
49             }
50         }
51     }
52 }
53
54 // x is a selected row node, not a column header.
55 void remove(int x)
56 {
57     for (int i = a[x].d; i != x; i = a[i].d)
58     {
59         a[a[i].l].r = a[i].r;
60         a[a[i].r].l = a[i].l;
61     }
62 }

```

逆序恢复与可采纳下界（接上页同一结构体）：

```
64 void restore(int x)
65 {
66     for (int i = a[x].u; i != x; i = a[i].u)
67     {
68         a[a[i].l].r = i;
69         a[a[i].r].l = i;
70     }
71 }
72
73 int lower_bound()
74 {
75     fill(mark.begin(), mark.end(), 0);
76     for (int c = a[0].r; c; c = a[c].r) mark[c] = 1;
77     int bound = 0;
78     for (int c = a[0].r; c; c = a[c].r)
79     {
80         if (!mark[c]) continue;
81         bound++;
82         mark[c] = 0;
83         for (int i = a[c].d; i != c; i = a[i].d)
84             for (int j = a[i].r; j != i; j = a[j].r)
85                 mark[a[j].c] = 0;
86     }
87     return bound;
88 }
```

最优搜索与求解入口（接上页同一结构体）：

```
90     void dfs()
91     {
92         int depth = path.size(), c = a[0].r;
93         if (!c)
94         {
95             if (depth < best)
96             {
97                 best = depth;
98                 ans = path;
99             }
100            return;
101        }
102        if (lower_bound() >= best - depth) return;
103        for (int j = a[c].r; j; j = a[j].r)
104            if (s[j] < s[c]) c = j;
105        for (int i = a[c].d; i != c; i = a[i].d)
106        {
107            path.push_back(a[i].row);
108            remove(i);
109            for (int j = a[i].r; j != i; j = a[j].r) remove(j);
110            dfs();
111            for (int j = a[i].l; j != i; j = a[j].l) restore(j);
112            restore(i);
113            path.pop_back();
114        }
115    }
116
117    optional<vector<int>> solve()
118    {
119        path.clear();
120        ans.clear();
121        best = m + 1;
122        dfs();
123        if (best > m) return nullopt;
124        return ans;
125    }
126 };
```

接口用法演示：最少行重复覆盖（本地接口示例）

演示约定：读入 n 个稀疏集合和 m 列；每行先给元素数，再给互异的 0-based 列号。输出最少选行数及原 0-based 行号，使每列至少出现一次；无解输出 -1， $m=0$ 输出 0 与空方案。此为接口演示，不冒充线上模板题。

所需模板：MinimumCover（第 5 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://oi-wiki.org/search/dlx/>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     vector<vector<int>>> rows(n);
8     for (auto &row : rows)
9     {
10         int k;
11         cin >> k;
12         row.resize(k);
13         for (int &c : row) cin >> c;
14     }
15     MinimumCover cover(m, rows);
16     auto ans = cover.solve();
17     if (!ans) cout << -1 << '\n';
18     else
19     {
20         cout << ans->size() << '\n';
21         for (int row : *ans) cout << row << ' ';
22         cout << '\n';
23     }
24 }
```

第二章 离线算法

2.1 CDQ 分治

2.1.1 静态三维偏序计数

代码依赖：Fenwick (第 44 页, C.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

`count(points)` 返回每个原输入位置被其他点逐坐标 $\leq$ 的次数，重复坐标仍是不同点，排除自身。输入 `array<long long,3>`，完整 signed64 坐标，空输入返回空；输入不变。内部按三维字典序合并重合点，压缩第三维，CDQ 归并与 Fenwick 回滚。 $O(n \log^2(n+1))$ 时间、 $O(n)$ 空间、 $O(\log(n+1))$ 递归； n 须 $\leq \text{INT_MAX}/2$ 且内存可分配，保证共享 Fenwick 的 `int` 步进不溢出。仅静态无权重偏序，不含严格不等号、动态更新或 CDQ 递推。

```
1 struct Dominance3D
2 {
3     static vector<int> count(const vector<array<long long, 3>> &p)
4     {
5         assert(p.size() <= INT_MAX / 2);
6         int n = int(p.size());
7         vector<int> order(n), group(n), w;
8         iota(order.begin(), order.end(), 0);
9         sort(order.begin(), order.end(), [&](int i, int j) { return p[i] < p[j]; });
10        vector<array<long long, 3>> v;
11        for (int i : order)
12        {
13            if (v.empty() || v.back() != p[i])
14            {
15                v.push_back(p[i]);
16                w.push_back(0);
17            }
18            ++w.back();
19            group[i] = int(v.size()) - 1;
20        }
21        int m = int(v.size());
22        vector<long long> z;
23        for (auto q : v) z.push_back(q[2]);
24        sort(z.begin(), z.end());
25        z.erase(unique(z.begin(), z.end()), z.end());
26        struct Node
27        {
28            long long y;
29            int z, w, id;
30        };
31        vector<Node> a(m), tmp(m);
32        vector<int> cnt(m), ans(n);
33        for (int i = 0; i < m; ++i)
34            a[i] = {v[i][1], int(lower_bound(z.begin(), z.end(), v[i][2]) -
35                                     z.begin()) + 1, w[i], i};
36        Fenwick<int> bit(int(z.size()));
```

三维偏序的 CDQ 归并与原序返回（接上页同一 count 函数）：

```

37     auto cdq = [&](auto &&self, int l, int r) -> void
38     {
39         if (r - l <= 1) return;
40         int mid = l + (r - l) / 2;
41         self(self, l, mid);
42         self(self, mid, r);
43         int i = l, k = 1;
44         for (int j = mid; j < r; ++j)
45         {
46             while (i < mid && a[i].y <= a[j].y)
47             {
48                 bit.add(a[i].z, a[i].w);
49                 tmp[k++] = a[i++];
50             }
51             cnt[a[j].id] += bit.sum(a[j].z);
52             tmp[k++] = a[j];
53         }
54         for (int t = l; t < i; ++t) bit.add(a[t].z, -a[t].w);
55         while (i < mid) tmp[k++] = a[i++];
56         copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
57     };
58     cdq(cdq, 0, m);
59     for (int i = 0; i < n; ++i)
60         ans[i] = cnt[group[i]] + w[group[i]] - 1;
61     return ans;
62 }
63 };

```

模板题使用：Luogu P3810 【模板】三维偏序 / 陌上花开

题意： $1 \leq n \leq 100000$ ， $1 \leq$ 三个坐标 $\leq k \leq 200000$ 。对每个输入点统计其他点三个坐标均 $\leq$ 自身的次数，输出次数为 $0..n-1$ 的点数直方图。重复位置分别计数且各自排除自己。 k 只约束题面坐标，库内部压缩，无需外部初始化；空输入和 signed64 极值另属 API 扩展。

所需模板：Fenwick（第 44 页，C.1 节）、Dominance3D（第 10 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3810>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, k;
6     cin >> n >> k;
7     vector<array<long long, 3>> p(n);
8     for (auto &[x, y, z] : p) cin >> x >> y >> z;
9     vector<int> hist(n);
10    for (int x : Dominance3D::count(p)) ++hist[x];
11    for (int x : hist) cout << x << '\n';
12 }

```

2.1.2 三维最长链、计数与方案

Chain3D(points, mod, strict=true), 坐标 array<long long,3>, 输入不变。

mod 为 1..INT_MAX, $n \leq \text{INT_MAX}/2$ 且内存可分配。

strict=true 要求每一步三个坐标都严格增加; false 为三坐标不减, 并固定按 (x,y,z, 原输入编号) 递增的规范顺序, 重合点可依编号串联, 不把相同坐标的排列重复计数。length/count 为最长链长度/模方案数; 空输入为 0 和 1%mod。len/ways/parent 按 0-based 原输入编号存每点结尾的最优状态与前驱, 无前驱为-1; path() 返回一条最长链原编号。并列时逐步选编号较小的前驱/终点, 不保证整条路径字典序最小; 计数为 0 模 mod 不表示无方案。CDQ 左递归 → 转移 → 右递归, 严格模式整组相同 x 不互转; y 扫描、z 压缩树状数组维护最大长度和同长计数。 $O(n \log^2(n+1))$ 时间、 $O(n)$ 空间、 $O(\log(n+1))$ 递归; 坐标只比较, 支持完整 signed64。所有成员只读, 重新赋值可重算。

```

1 struct Chain3D
2 {
3     using Point = array<long long, 3>;
4
5     struct State
6     {
7         int len = 0, ways = 0, end = -1;
8     };
9
10    vector<int> len, ways, parent;
11    int length = 0, count = 0, last = -1;
12
13    Chain3D(const vector<Point> &p, int mod, bool strict = true)
14    {
15        assert(mod > 0 && p.size() <= INT_MAX / 2);
16        int n = p.size();
17        count = 1 % mod;
18        len.assign(n, 1);
19        ways.assign(n, 1 % mod);
20        parent.assign(n, -1);
21        if (!n) return;

```


坐标顺序、分组与状态合并（接上页同一结构体）：

```

22     vector<int> order(n), z(n), cut{0};
23     vector<long long> values;
24     iota(order.begin(), order.end(), 0);
25     sort(order.begin(), order.end(), [&](int x, int y)
26     {
27         if (p[x] != p[y]) return p[x] < p[y];
28         return x < y;
29     });
30     for (int i = 1; i < n; i++)
31     {
32         if (!strict || p[order[i]][0] != p[order[i - 1]][0]) cut.push_back(i);
33     }
34     cut.push_back(n);
35     for (auto q : p) values.push_back(q[2]);
36     sort(values.begin(), values.end());
37     values.erase(unique(values.begin(), values.end()), values.end());
38     for (int i = 0; i < n; i++)
39     {
40         z[i] = lower_bound(values.begin(), values.end(), p[i][2]) - values.begin() + 1;
41     }
42     vector<State> bit(values.size() + 1);
43     auto join = [&](State x, State y)
44     {
45         if (x.len != y.len) return x.len > y.len ? x : y;
46         x.ways = ((long long)x.ways + y.ways) % mod;
47         x.end = min(x.end, y.end);
48         return x;
49     };

```

先转移再右递归（同一构造函数内）（接上页同一结构体）：

```

50     auto cdq = [&](auto &&self, int l, int r) -> void
51     {
52         if (r - l <= 1) return;
53         int mid = l + (r - l) / 2;
54         self(self, l, mid);
55         {
56             vector<int> a(order.begin() + cut[l], order.begin() + cut[mid]);
57             vector<int> b(order.begin() + cut[mid], order.begin() + cut[r]);
58             auto by_y = [&](int x, int y)
59             {
60                 return p[x][1] < p[y][1];
61             };
62             sort(a.begin(), a.end(), by_y);
63             sort(b.begin(), b.end(), by_y);
64             int i = 0;
65             for (int v : b)
66             {
67                 while (i < (int)a.size())
68                 {
69                     int u = a[i];
70                     if (strict ? p[u][1] >= p[v][1] : p[u][1] > p[v][1]) break;
71                     i++;
72                     for (int t = z[u]; t < (int)bit.size(); t += t & -t)
73                     {
74                         bit[t] = join(bit[t], {len[u], ways[u], u});
75                     }
76                 }
77                 State best;
78                 for (int t = z[v] - strict; t > 0; t -= t & -t)
79                 {
80                     best = join(best, bit[t]);
81                 }
82                 if (!best.len) continue;
83                 if (best.len + 1 > len[v])
84                 {
85                     len[v] = best.len + 1;
86                     ways[v] = best.ways;
87                     parent[v] = best.end;
88                 }
89                 else if (best.len + 1 == len[v])
90                 {
91                     ways[v] = ((long long)ways[v] + best.ways) % mod;
92                     parent[v] = min(parent[v], best.end);
93                 }
94             }
95             for (int j = 0; j < i; j++)
96             {
97                 for (int t = z[a[j]]; t < (int)bit.size(); t += t & -t) bit[t] = State{};
98             }
99         }
100         self(self, mid, r);
101     };

```

汇总答案与路径恢复（接上页同一结构体）：

```

102     cdq(cdq, 0, cut.size() - 1);
103     State best;
104     for (int i = 0; i < n; i++) best = join(best, {len[i], ways[i], i});
105     length = best.len;
106     count = best.ways;
107     last = best.end;
108 }
109
110 vector<int> path() const
111 {
112     vector<int> ans;
113     for (int u = last; u != -1; u = parent[u]) ans.push_back(u);
114     reverse(ans.begin(), ans.end());
115     return ans;
116 }
117 };

```

接口用法演示：三维最长链与原编号方案（接口演示）

演示约定：输入 $n, \text{mod}, \text{strict}$ 及 n 个三维整数点。 $\text{strict}=1$ 为三维都严格增加； $\text{strict}=0$ 为三维不减，重合点按原编号串联，不重复计算它们的排列。输出全局长度/模方案数、一条原编号路径，以及各原输入点的结尾长度/模方案数/前驱编号。空输入全局为 0 和 $1\% \text{mod}$ ，路径为空； $\text{mod}=1$ 时计数 0 也能恢复路径。

所需模板：Chain3D（第 12 页，2.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://oi-wiki.org/misc/cdq-divide/>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, mod, strict;
6     cin >> n >> mod >> strict;
7     vector<Chain3D::Point> p(n);
8     for (auto &v : p) cin >> v[0] >> v[1] >> v[2];
9     Chain3D chain(p, mod, strict);
10    cout << chain.length << ' ' << chain.count << '\n';
11    for (int id : chain.path()) cout << id << ' ';
12    cout << '\n';
13    for (int i = 0; i < n; i++)
14    {
15        cout << chain.len[i] << ' ' << chain.ways[i] << ' ' << chain.parent[i] << '\n';
16    }
17 }

```

接口用法演示：三维弱链计数 (kuangbin HDU4742 源码模型)

演示约定：输入 T 组，每组给 $n \geq 1$ 和 n 个三维整数点，输出按三维弱偏序及重合点固定顺序得到的最长链长度、方案数模 2^{30} 。这是原稿源码的闭边界模型；其文字描述却写三个严格小于，原 HDU4742 题面/约束/协议尚未核验，不视为正式模板题或已完成 OJ 适配。

所需模板：Chain3D (第 12 页, 2.1.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源： <https://github.com/kuangbin/ACM-ICPC>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int tests;
6     cin >> tests;
7     while (tests--)
8     {
9         int n;
10        cin >> n;
11        vector<Chain3D::Point> p(n);
12        for (auto &v : p) cin >> v[0] >> v[1] >> v[2];
13        Chain3D chain(p, 1 << 30, false);
14        cout << chain.length << ' ' << chain.count << '\n';
15    }
16 }
```

2.2 整体二分

2.2.1 整体二分：动态区间第 k 小

代码依赖：Fenwick (第 44 页, C.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

OverallKth(a) 读取 0-based 非空初始数组；后续位置用 1..n。set(pos,value) 登记单点赋值；query(l,r,k) 登记闭区间第 k 小并返回 0-based 查询 ID， $1 \leq k \leq r-l+1$ 。run() 按查询 ID 返回 `vector<long long>`，答案对应登记查询时的状态，不是最后数组；可重复 run，也可继续登记后重新 run。支持完整 long long 值域，不做值的算术；同值修改跳过。依赖 Fenwick<int>，递归值域分治，保持时间顺序并撤销贡献。 $n \leq \text{INT_MAX}/2$ ，事件数 $E=n+2U+Q \leq \text{INT_MAX}$ (U 为实际变值次数)，另须满足内存限制。V 为出现过的不同值数，时间 $O(E \log(E+1) + E \log(n+1) \log(V+1))$ ，空间 $O(n+E)$ 。只读 current/events/queries，不外部修改。

```

1 struct OverallKth
2 {
3     using ll = long long;
4
5     struct Event
6     {
7         int l, r, k, id;
8         ll value;
9     };
10
11     int n, queries = 0;
12     vector<ll> current;
13     vector<Event> events;
14
15     OverallKth(const vector<ll> &a) : n(a.size()), current(a)
16     {
17         assert(n > 0);
18         for (int i = 1; i <= n; i++)
19         {
20             events.push_back({i, 0, 1, -1, a[i - 1]});
21         }
22     }
23
24     void set(int pos, ll value)
25     {
26         assert(1 <= pos && pos <= n);
27         if (current[pos - 1] == value) return;
28         events.push_back({pos, 0, -1, -1, current[pos - 1]});
29         events.push_back({pos, 0, 1, -1, value});
30         current[pos - 1] = value;
31     }
32
33     int query(int l, int r, int k)
34     {
35         assert(1 <= l && l <= r && r <= n);
36         assert(1 <= k && k <= r - l + 1);
37         int id = queries++;
38         events.push_back({l, r, k, id, 0});
39         return id;
40     }

```

继续求解（接上页同一结构体）：

```
42     vector<ll> run() const
43     {
44         vector<ll> answer(queries), values;
45         if (queries == 0) return answer;
46         auto a = events;
47         for (auto e : a)
48         {
49             if (e.id < 0) values.push_back(e.value);
50         }
51         sort(values.begin(), values.end());
52         values.erase(unique(values.begin(), values.end()), values.end());
53         for (auto &e : a)
54         {
55             if (e.id < 0)
56             {
57                 e.value = lower_bound(values.begin(), values.end(), e.value) - values.begin();
58             }
59         }
60         vector<Event> buffer(a.size());
61         vector<bool> left(a.size());
62         Fenwick<int> bit(n);
```

继续求解（接上页同一结构体）：

```

63     auto solve = [&](auto &&self, int lo, int hi, int begin, int end) -> void
64     {
65         if (begin == end) return;
66         if (lo == hi)
67         {
68             for (int i = begin; i < end; i++)
69             {
70                 if (a[i].id >= 0) answer[a[i].id] = values[lo];
71             }
72             return;
73         }
74         int mid = lo + (hi - lo) / 2, count = 0;
75         for (int i = begin; i < end; i++)
76         {
77             auto &e = a[i];
78             if (e.id < 0)
79             {
80                 left[i] = e.value <= mid;
81                 if (left[i]) bit.add(e.l, e.k);
82             }
83             else
84             {
85                 int smaller = bit.query(e.l, e.r);
86                 left[i] = e.k <= smaller;
87                 if (!left[i]) e.k -= smaller;
88             }
89             count += left[i];
90         }
91         int p = begin, q = begin + count;
92         for (int i = begin; i < end; i++)
93         {
94             auto e = a[i];
95             if (e.id < 0 && left[i]) bit.add(e.l, -e.k);
96             if (left[i]) buffer[p++] = e;
97             else buffer[q++] = e;
98         }
99         copy(buffer.begin() + begin, buffer.begin() + end, a.begin() + begin);
100        self(self, lo, mid, begin, begin + count);
101        self(self, mid + 1, hi, begin + count, end);
102    };
103    solve(solve, 0, int(values.size()) - 1, 0, a.size());
104    return answer;
105 }
106 };

```

应用题补充：P2617 Dynamic Rankings（整体二分）

题意： $n, m \leq 100000$ ，初值和修改值 $0..1e9$ ； $Q\ l\ r\ k$ 查询闭区间第 k 小， $C\ pos\ value$ 单点赋值。先按原时间顺序登记，最后统一求解输出；不继承在线 DynamicKth 的 AC。

所需模板：Fenwick（第 44 页，C.1 节）、OverallKth（第 17 页，2.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P2617>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     vector<long long> a(n);
8     for (auto &x : a) cin >> x;
9     OverallKth solver(a);
10    for (int i = 0; i < m; i++)
11    {
12        char op;
13        cin >> op;
14        if (op == 'Q')
15        {
16            int l, r, k;
17            cin >> l >> r >> k;
18            solver.query(l, r, k);
19        }
20        else
21        {
22            int pos;
23            long long value;
24            cin >> pos >> value;
25            solver.set(pos, value);
26        }
27    }
28    for (long long value : solver.run()) cout << value << '\n';
29 }
```


2.3 莫队算法

2.3.1 带修改莫队

2.3.1.1 带修改的莫队

ModifiedMo(n, m) 保存离线询问, $n > 0$, m 为修改总数。add(l, r, t) 登记第 t 次修改完成后的 $[l, r]$, 返回从 0 开始的询问编号; $0 \leq l \leq r \leq n$, $0 \leq t \leq m$, 询问数小于 INT_MAX。run(add, del, chg, ans, block=0) 排序并移动三个指针。add(p)/del(p) 按下标更新当前区间统计; chg($i, forward, l, r$) 将编号 i 的修改前进或回退, 并同步修正当前 $[l, r]$ 内的贡献; ans(id) 记录结果。修改编号从 0 开始, 前进第 i 次修改时原时间为 i , 回退时原时间为 $i+1$ 。回调初始状态必须为原数组、空区间、时间 0; 重复 run 前由调用方重置状态。状态不会自动恢复, 原询问顺序不变。仅适用于可增删维护的区间统计, 需区分左右端操作时另行拆分回调。默认块长约 $n^{2/3}$, 也可显式指定正块长。 n 、询问数、修改数同阶且回调 $O(1)$ 时总时间 $O(n^{5/3})$, 调度器空间 $O(q)$ 。

```

1 struct ModifiedMo
2 {
3     struct Query
4     {
5         int l, r, t, id;
6     };
7
8     int n, m;
9     vector<Query> q;
10
11     ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }
12
13     int add(int l, int r, int t)
14     {
15         assert(0 <= l && l <= r && r <= n);
16         assert(0 <= t && t <= m);
17         int id = q.size();
18         q.push_back({l, r, t, id});
19         return id;
20     }

```

```

22     template <class Add, class Del, class Chg, class Ans>
23     void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
24     {
25         if (q.empty()) return;
26         if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
27         assert(block > 0);
28         auto ord = q;
29         auto cmp = [&](const Query &a, const Query &b)
30         {
31             int x = a.l / block, y = b.l / block;
32             if (x != y) return x < y;
33             int p = a.r / block, q = b.r / block;
34             if (p != q) return (x & 1) ? p > q : p < q;
35             return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
36         };
37         sort(ord.begin(), ord.end(), cmp);
38         int l = 0, r = 0, t = 0;
39         for (auto q : ord)
40         {
41             while (t < q.t)
42             {
43                 chg(t, true, l, r);
44                 t++;
45             }
46             while (t > q.t)
47             {
48                 t--;
49                 chg(t, false, l, r);
50             }
51             while (l > q.l) add(--l);
52             while (r < q.r) add(r++);
53             while (l < q.l) del(l++);
54             while (r > q.r) del(--r);
55             ans(q.id);
56         }
57     }
58 };

```

模板题使用：Luogu P1903

题意：单点改色、区间不同颜色数， $n, m \leq 133333$ ，颜色 $1..1000000$ 。先读完操作，`cur` 仅用于记录每次修改前后的颜色，`a` 保持初始数组。题目闭区间 $[L, R]$ 转为 $[L-1, R)$ ，询问时间是此前修改数。`chg` 必须先删旧贡献、改 `a[p]`、再加新贡献，回退使用 `pre`。按登记顺序输出。三维莫队，回调 $O(1)$ ，同阶规模时 $O(n^{5/3})$ 。

所需模板：ModifiedMo（第 21 页，2.3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P1903>

Luogu P1903: 使用代码

```

1 int main()
2 {
3     int n, m;
4     scanf("%d%d", &n, &m);
5     vector<int> a(n);
6     for (int &v : a) scanf("%d", &v);
7     vector<int> cur = a;
8     vector<array<int, 3>> upd;
9     vector<array<int, 3>> qry;
10    for (int i = 0; i < m; i++)
11    {
12        char op;
13        int x, y;
14        scanf(" %c%d%d", &op, &x, &y);
15        x--;
16        if (op == 'Q')
17            qry.push_back({x, y, int(upd.size())});
18        else
19        {
20            upd.push_back({x, cur[x], y});
21            cur[x] = y;
22        }
23    }
24    ModifiedMo mo(n, upd.size());
25    for (auto [l, r, t] : qry) mo.add(l, r, t);
26    vector<int> cnt(1000001), res(qry.size());
27    int tot = 0;
28    auto add = [&](int p)
29    {
30        if (cnt[a[p]]++ == 0) tot++;
31    };
32    auto del = [&](int p)
33    {
34        if (--cnt[a[p]] == 0) tot--;
35    };
36    auto chg = [&](int t, bool fwd, int l, int r)
37    {
38        auto [p, pre, nxt] = upd[t];
39        bool in = l <= p && p < r;
40        if (in) del(p);
41        a[p] = fwd ? nxt : pre;
42        if (in) add(p);
43    };
44    auto ans = [&](int id)
45    {
46        res[id] = tot;
47    };
48    mo.run(add, del, chg, ans);
49    for (int v : res) printf("%d\n", v);
50 }

```

2.3.2 树上莫队

2.3.2.1 树上莫队：点权与边权

代码依赖：HLD（第 59 页，E.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

先对非空连通树 `HLD.build(root)`，再 `TreeMo(tree)`；`tree` 须活到 `run` 结束且不能修改。点号 `1..n`，双欧拉序长 `2n`，`n ≤ INT_MAX/2`；DFS 递归。`add(u,v,edge=false)` 返回 0-based 查询 ID；默认点权，`edge=true` 把父边权记在子点并排除 LCA。`run(add,del,ans,block=0)` 的 `add/del` 参数是顶点，`ans` 参数是查询 ID；只适合与集合排列无关且可增删的统计。调用前统计为空，结束恢复空；不可在回调内修改树/查询，可重复运行。混用点边查询时回调须有一致的数据解释。默认块长 $\max(1, 2n/\sqrt{m})$ ；自定 $B > 0$ 。 $m > 0$ 时调度 $O(m \log(m+1) + mB + n^2/B + n)$ ，查询转换另需 $O(m \log n)$ ，回调代价另计。空间 $O(n+m)$ ，复用 HLD；整数大小另须满足内存限制。

```

1 struct TreeMo
2 {
3     struct Query
4     {
5         int l, r, extra, id;
6     };
7
8     const HLD &tree;
9     vector<int> in, out, euler;
10    vector<Query> q;
11
12    TreeMo(const HLD &tree) : tree(tree), in(tree.n + 1), out(tree.n + 1)
13    {
14        assert(tree.n > 0 && tree.timer == tree.n);
15        dfs(tree.rk[1], 0);
16    }
17
18    void dfs(int u, int p)
19    {
20        in[u] = euler.size();
21        euler.push_back(u);
22        for (int v : tree.g[u])
23        {
24            if (v == p) continue;
25            dfs(v, u);
26        }
27        out[u] = euler.size();
28        euler.push_back(u);
29    }
30
31    int add(int u, int v, bool edge = false)
32    {
33        assert(1 <= u && u <= tree.n && 1 <= v && v <= tree.n);
34        if (in[u] > in[v]) swap(u, v);
35        int w = tree.lca(u, v);
36        int l = w == u ? in[u] : out[u];
37        int extra = w == u ? 0 : w;
38        if (edge) extra = w == u ? w : 0;
39        int id = q.size();
40        q.push_back({l, in[v] + 1, extra, id});
41        return id;
42    }

```

查询调度与状态恢复（接上页同一结构体）：

```

44     template <class Add, class Del, class Ans>
45     void run(Add add, Del del, Ans ans, int block = 0) const
46     {
47         if (q.empty()) return;
48         int n = euler.size();
49         if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
50         assert(block > 0);
51         auto ord = q;
52         sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
53         {
54             int x = a.l / block, y = b.l / block;
55             if (x != y) return x < y;
56             return (x & 1) ? a.r > b.r : a.r < b.r;
57         });
58         vector<bool> active(tree.n + 1);
59         auto toggle = [&](int u)
60         {
61             if (active[u]) del(u);
62             else add(u);
63             active[u] = !active[u];
64         };
65         int l = 0, r = 0;
66         for (auto a : ord)
67         {
68             while (l > a.l) toggle(euler[--l]);
69             while (r < a.r) toggle(euler[r++]);
70             while (l < a.l) toggle(euler[l++]);
71             while (r > a.r) toggle(euler[--r]);
72             if (a.extra) toggle(a.extra);
73             ans(a.id);
74             if (a.extra) toggle(a.extra);
75         }
76         for (int u = 1; u <= tree.n; u++)
77         {
78             if (active[u]) del(u);
79         }
80     }
81 };

```

应用题补充：SP10707 COT2 - Count on a tree II

题意：静态树路径不同点权数， $n \leq 40000, m \leq 100000$ ；离散化权值，双欧拉序维护出现次数，额外补 LCA。
竞赛应用；边权语义另由核心对拍验证。

所需模板：HLD（第 59 页，E.1.1 节）、TreeMo（第 24 页，2.3.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/SP10707>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     vector<long long> a(n + 1), values;
8     for (int u = 1; u <= n; u++)
9     {
10         cin >> a[u];
11         values.push_back(a[u]);
12     }
13     sort(values.begin(), values.end());
14     values.erase(unique(values.begin(), values.end()), values.end());
15     vector<int> color(n + 1), count(values.size());
16     for (int u = 1; u <= n; u++)
17     {
18         color[u] = lower_bound(values.begin(), values.end(), a[u]) - values.begin();
19     }
20     HLD tree(n);
21     for (int i = 1; i < n; i++)
22     {
23         int u, v;
24         cin >> u >> v;
25         tree.add(u, v);
26     }
27     tree.build();
28     TreeMo mo(tree);
29     for (int i = 0; i < m; i++)
30     {
31         int u, v;
32         cin >> u >> v;
33         mo.add(u, v);
34     }
```

继续处理操作（接上页同一 main）：

```
35     vector<int> answer(m);
36     int distinct = 0;
37     auto add = [&](int u)
38     {
39         if (count[color[u]]++ == 0) distinct++;
40     };
41     auto del = [&](int u)
42     {
43         if (--count[color[u]] == 0) distinct--;
44     };
45     mo.run(add, del, [&](int id)
46     {
47         answer[id] = distinct;
48     });
49     for (int x : answer) cout << x << '\n';
50 }
```

2.3.3 回滚莫队

2.3.3.1 回滚莫队

RollbackMo(n), $n \geq 0$; add(l, r) 登记 0-based 半开区间 $[l, r)$, 允许空区间, 返回 0-based 查询 ID。run(add, snapshot, rollback, ans, block=0): add 接收位置, snapshot 返回当前快照, rollback 恢复该祖先快照, ans 接收查询 ID。开始时统计必须为空; 结束恢复初始快照。每次 add 都须可撤销, 包含长期右段; 不保留失效快照、不修改查询。只支持与插入次序无关的区间统计。默认 $B = \max(1, n/\sqrt{m})$; $m > 0$ 时调度与 $O(1)$ 增删总计 $O(m \log(m+1) + mB + n^2/B + n)$, 空间 $O(m)$ 加回调状态/历史 $O(n)$ 。可重复 run, $n=0$ 只允许空查询; 无查询不调用回调。

```

1 struct RollbackMo
2 {
3     struct Query
4     {
5         int l, r, id;
6     };
7
8     int n;
9     vector<Query> q;
10
11     RollbackMo(int n) : n(n)
12     {
13         assert(n >= 0);
14     }
15
16     int add(int l, int r)
17     {
18         assert(0 <= l && l <= r && r <= n);
19         int id = q.size();
20         q.push_back({l, r, id});
21         return id;
22     }
23
24     template <class Add, class Snap, class Undo, class Ans>
25     void run(Add add, Snap snapshot, Undo rollback, Ans ans, int block = 0) const
26     {
27         if (q.empty()) return;
28         if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
29         assert(block > 0);
30         auto ord = q;
31         sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
32         {
33             int x = a.l / block, y = b.l / block;
34             if (x != y) return x < y;
35             return a.r < b.r;
36         });

```


查询调度与状态恢复（接上页同一结构体）：

```
37     auto empty = snapshot();
38     int last = -1, r = 0;
39     for (auto a : ord)
40     {
41         int b = a.l / block;
42         int end = a.l + min(block - a.l % block, n - a.l);
43         if (b != last)
44         {
45             rollback(empty);
46             last = b;
47             r = end;
48         }
49         if (a.r <= end)
50         {
51             auto saved = snapshot();
52             for (int i = a.l; i < a.r; i++) add(i);
53             ans(a.id);
54             rollback(saved);
55             continue;
56         }
57         while (r < a.r) add(r++);
58         auto saved = snapshot();
59         for (int i = end; i > a.l; ) add(--i);
60         ans(a.id);
61         rollback(saved);
62     }
63     rollback(empty);
64 }
65 };
```

模板题使用：P5906 【模板】回滚莫队 & 不删除莫队

题意： $n, m \leq 200000$ ，值 $1..2e9$ ；对每个 1-based 闭区间输出相同值的最远下标差，无重复输出 0。离散化，保存各值最左/最右位置和全局答案，历史逐次恢复。

所需模板：RollbackMo（第 28 页，2.3.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P5906>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     vector<int> a(n), values;
8     for (int &x : a) cin >> x;
9     values = a;
10    sort(values.begin(), values.end());
11    values.erase(unique(values.begin(), values.end()), values.end());
12    for (int &x : a)
13    {
14        x = lower_bound(values.begin(), values.end(), x) - values.begin();
15    }
16    int m;
17    cin >> m;
18    RollbackMo mo(n);
19    for (int i = 0; i < m; i++)
20    {
21        int l, r;
22        cin >> l >> r;
23        mo.add(l - 1, r);
24    }
```

继续处理操作（接上页同一 main）：

```
25     struct Change
26     {
27         int color, first, last, best;
28     };
29     vector<Change> history;
30     vector<int> first(values.size(), n), last(values.size(), -1), answer(m);
31     int best = 0;
32     auto add = [&](int i)
33     {
34         int c = a[i];
35         history.push_back({c, first[c], last[c], best});
36         first[c] = min(first[c], i);
37         last[c] = max(last[c], i);
38         best = max(best, last[c] - first[c]);
39     };
40     auto snapshot = [&]()
41     {
42         return history.size();
43     };
44     auto rollback = [&](size_t saved)
45     {
46         while (history.size() > saved)
47         {
48             auto h = history.back();
49             history.pop_back();
50             first[h.color] = h.first;
51             last[h.color] = h.last;
52             best = h.best;
53         }
54     };
55     mo.run(add, snapshot, rollback, [&](int id)
56     {
57         answer[id] = best;
58     });
59     for (int x : answer) cout << x << '\n';
60 }
```

2.3.4 莫队二次离线

2.3.4.1 莫队二次离线：异或位数配对

`xor_hamming_pairs(a, queries, k, bits=14)` 按查询输入顺序返回 long long 答案, 统计 $1 \leq i < j < r$ 且 $\text{popcount}(a[i] \oplus a[j]) = k$ 的下标对。queries 为 0-based 半开区间, 允许空区间、重复查询与空数组; $0 \leq \text{bits} \leq 20$, $0 \leq a[i] < 2^{\text{bits}}$, n , m 和容器下标须可由 int 表示。k 超出 $[0, \text{bits}]$ 直接返回全零; 相等值来自不同下标才构成一对。莫队移动生成前缀事件, 第二次离线按前缀扫描; $k=0$ 显式扣掉自身。令 $C = \text{binom}(\text{bits}, k)$, 时间 $O(2^{\text{bits}} + nC + m \log(m+1) + n\sqrt{m+m})$, 空间 $O(2^{\text{bits}} + n + m)$, $m=0$ 直接返回。无需维护可逆的在线窗口, 也不支持修改。

```

1 // Queries are 0-based half-open; count unordered pairs of distinct indices.
2 vector<long long> xor_hamming_pairs(const vector<int> &a,
3                                     const vector<pair<int, int>> &queries,
4                                     int k,
5                                     int bits = 14)
6 {
7     int n = a.size(), m = queries.size();
8     vector<long long> answer(m);
9     assert(0 <= bits && bits <= 20);
10    int universe = 1 << bits;
11    for (int x : a) assert(0 <= x && x < universe);
12    for (auto [l, r] : queries) assert(0 <= l && l <= r && r <= n);
13    if (!m || k < 0 || k > bits) return answer;
14    vector<int> masks;
15    for (int x = 0; x < universe; x++)
16        if (popcount(unsigned(x)) == k) masks.push_back(x);
17    vector<int> count(universe);
18    vector<long long> prefix(n + 1);
19    for (int i = 0; i < n; i++)
20    {
21        prefix[i + 1] = prefix[i] + count[a[i]];
22        for (int mask : masks) count[a[i] ^ mask]++;
23    }
24    int block = max(1, int(n / sqrt(m)));
25    vector<int> order(m);
26    iota(order.begin(), order.end(), 0);
27    sort(order.begin(),
28         order.end(),
29         [&](int x, int y)
30         {
31             int bx = queries[x].first / block, by = queries[y].first / block;
32             if (bx != by) return bx < by;
33             return bx & 1 ? queries[x].second > queries[y].second
34                 : queries[x].second < queries[y].second;
35         });
36
37    struct Event
38    {
39        int id, l, r, sign;
40    };
41
42    vector<vector<Event>> events(n + 1);

```

莫队端点移动与前缀事件（接上页同一函数）：

```
43     int l = 0, r = 0;
44     for (int id : order)
45     {
46         auto [ql, qr] = queries[id];
47         if (r < qr)
48         {
49             events[l].push_back({id, r, qr, -1});
50             answer[id] += prefix[qr] - prefix[r];
51             r = qr;
52         }
53         if (l > ql)
54         {
55             events[r].push_back({id, ql, l, 1});
56             answer[id] -= prefix[l] - prefix[ql];
57             l = ql;
58         }
59         if (r > qr)
60         {
61             events[l].push_back({id, qr, r, 1});
62             answer[id] -= prefix[r] - prefix[qr];
63             r = qr;
64         }
65         if (l < ql)
66         {
67             events[r].push_back({id, l, ql, -1});
68             answer[id] += prefix[ql] - prefix[l];
69             l = ql;
70         }
71     }
```

第二次离线与差分还原（接上页同一函数）：

```

72     fill(count.begin(), count.end(), 0);
73     for (int p = 0; p <= n; p++)
74     {
75         if (p)
76             for (int mask : masks) count[a[p - 1] ^ mask]++;
77         for (auto e : events[p])
78             for (int i = e.l; i < e.r; i++)
79             {
80                 int value = count[a[i]] - (k == 0 && i < p);
81                 answer[e.id] += 1LL * e.sign * value;
82             }
83     }
84     long long total = 0;
85     for (int id : order)
86     {
87         total += answer[id];
88         answer[id] = total;
89     }
90     return answer;
91 }

```

模板题使用：Luogu P4887

题意：统计每个区间内异或值恰有 k 个 1 的无序下标对。输入 1-based 闭区间 $[l, r]$ ，只将 1 减一转为 0-based 半开区间； k 可以大于 14，此时全零。 $k=0$ 统计相等值的不同下标，不计自身。

所需模板：`xor_hamming_pairs`（第 32 页，2.3.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P4887>

```

1  int main()
2  {
3      ios::sync_with_stdio(false);
4      cin.tie(nullptr);
5      int n, m, k;
6      cin >> n >> m >> k;
7      vector<int> a(n);
8      for (int &x : a) cin >> x;
9      vector<pair<int, int>> queries(m);
10     for (auto &[l, r] : queries)
11     {
12         cin >> l >> r;
13         l--;
14     }
15     for (long long x : xor_hamming_pairs(a, queries, k)) cout << x << '\n';
16 }

```

第三章 配套知识与验证范围

3.1 带修莫队：时间轴与区间状态

先离线读完操作，记录每个单点修改的下标、修改前值和修改后值。询问的时间 t 表示前 t 次修改已执行；输入中的询问本身不推进时间。同一位置的第二次修改，其旧值必须来自第一次修改后的数组，不能始终使用初始值。需要离散化时，应将全部初值与修改后值一同纳入。

调度器保持三个指针 (l, r, t) ，回调统计恰好对应当前版本的半开区间 $[l, r)$ 。前进修改 i 时写入 after，回退时写入 before。若修改点在当前区间内，必须先删除旧贡献、改值、再加入新贡献；区间外只改数组。随后移动区间端点，最后按原询问编号写答案。扩展区间先于收缩，支持空区间和互不相交区间。整个 run 结束后状态停留在排序后的最后一条询问；再次调用前须重置原数组和统计量。

左端点分块、右端点分块，再按时间排序，相邻块采用蛇形顺序。时间关键字是真实修改次数，不能把询问编号当作时间维度。令块长为 B ，询问数为 Q ，修改数为 U ，则常数时间回调下的移动量上界为

$$O\left(QB + \frac{n^2}{B} + U \min(Q, (\lfloor n/B \rfloor + 1)^2)\right).$$

另有 $O(Q \log Q)$ 排序与 $O(Q)$ 调度空间。三者同阶时取 $B \approx n^{2/3}$ ，得到 $O(n^{5/3})$ ；规模悬殊时可手动调整块长。回调需要区分左右端增删时应拆分四个方向，当前简版适合不同数、频次矩等统计。

P1903 数颜色 驱动以颜色频次数组维护不同数，真实操作顺序建立修改前后值，随后交给调度器。本地快照测试逐项检查每个时间、每个区间及修改回退状态，另以顺序 set 查询核对完整驱动。最大规模验证 133333 个元素和操作，并覆盖仅修改、仅询问与重复修改同一位置。最终简短命名版本在 **P1903 记录 297653743** 通过全部 13 个测试，最慢单点 546 ms、峰值内存 14.65 MB。此记录使用默认块长与不同颜色数统计；自定义块长、空区间及任意登记时间由本地测试覆盖。最快提交榜的排名及总数尚未核验。

3.2 莫队二次离线：前缀事件与自身扣除

对区间内满足 $H(i, j) = [\text{popcount}(a_i \text{ xor } a_j) = k]$ 的无序下标对，使用 `xor_hamming_pairs`。令 $t(i) = \sum_{j < i} H(i, j)$, $P(x) = \sum_{i < x} t(i)$, 以及 $T(p, i) = \sum_{j < p} H(i, j) - [k = 0 \wedge i < p]$ 。右端扩展时贡献为 $t(i) - T(L, i)$, 左端扩展为 $T(R, i) - t(i)$; 删除取相反数。整段端点移动的 t 和由 P 的区间差求出, 剩余部分存成按前缀长度分桶的事件。第二遍顺扫只做插入, 每次枚举异或后有 k 个置位的掩码更新计数; 事件中的每个下标只读一次计数, 并在 $k = 0$ 时扣自身。最后按莫队顺序累加窗口答案差, 再按原编号返回。

区间统一为 0-based 半开区间, 先扩右、扩左, 再缩右、缩左, 保持窗口合法。允许空区间; 没有查询时直接返回。位宽默认为 14, 若 k 超出位宽则答案全零。P4887 的 $k < 2^{14}$ 并不代表 $k \leq 14$ 。令 $C = \binom{b}{k}$, 块长约为 $n/\sqrt{m}$, 总时间 $O(2^b + nC + m \log(m+1) + n\sqrt{m} + m)$, 空间 $O(2^b + n + m)$ 。端点移动不再乘枚举掩码数 C 。答案与前缀和使用 long long; 十万个相等数的配对数超过 int32。本地全枚举、随机逐对参考及十万规模的 $k = 0, 7$ 分支通过; 线上验证待补。

第一部分

跨类依赖

附录 A 数论

A.1 模算术简介

A.1.1 静态模整数

固定 int 模数 $1 \leq \text{mod} \leq \text{INT_MAX}$, 可为合数。构造接受 signed64 并正规化, v 保持 $0.. \text{mod}-1$; 不要手动破坏正规化。加减乘 $O(1)$, pow 非负 signed64 指数、时间 $O(\log(e+1))$ 。try_inv 用欧几里得求逆: $\text{gcd}(v, \text{mod})=1$ 时返回 optional 值, 否则 nullopt; $\text{mod}=1$ 返回代表 0。inv、/、/= 要求分母可逆; 断言开启时诊断违反前提, 关闭断言不承诺拒绝非法除法。逆元 $O(\log(\text{mod}+1))$ 、额外空间 $O(1)$, 只用 long long, 不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用, 支持自赋值与链式运算, 可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
1 template <int mod> struct ModInt
2 {
3     static_assert(mod >= 1);
4     int v;
5
6     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
7
8     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
9
10    ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
11
12    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
13
14    ModInt pow(long long e) const
15    {
16        assert(e >= 0);
17        ModInt a = *this, r = 1;
18        for (; e >= 1, a = a * a)
19            if (e & 1) r = r * a;
20        return r;
21    }
```

求逆与除法（接上页同一结构体）：

```

23     optional<ModInt> try_inv() const
24     {
25         long long a = v, b = mod, x = 1, y = 0;
26         while (b)
27         {
28             long long q = a / b;
29             a -= q * b;
30             swap(a, b);
31             x -= q * y;
32             swap(x, y);
33         }
34         if (a != 1) return nullopt;
35         return ModInt(x);
36     }
37
38     ModInt inv() const
39     {
40         auto r = try_inv();
41         assert(r);
42         return *r;
43     } // requires an invertible residue
44
45     ModInt operator/(ModInt b) const { return *this * b.inv(); }
46
47     ModInt &operator+=(ModInt b) { return *this = *this + b; }
48
49     ModInt &operator-=(ModInt b) { return *this = *this - b; }
50
51     ModInt &operator*=(ModInt b) { return *this = *this * b; }
52
53     ModInt &operator/=(ModInt b) { return *this = *this / b; }
54 };

```

联合使用示例见第 43 页（Luogu P1313 计算系数），其中直接调用本模板。

A.2 卢卡斯定理

A.2.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p, choose(n,k) 返回 $C(n,k)$ 模 p; n、k 为完整 unsigned long long 范围, $k > n$ 返回 0, $C(0,0)=1$ 。重新赋值 Lucas(p) 可更换模数并重新预处理; 不检测素性, 要求调用者保证 p 为素数且 $p \geq 2$ 。预处理时间 $O(p + \log p)$, 动态版空间 $O(p)$; 单次查询 $O(1 + \log_p(n+1))$ 。适合小素数模数, 通常 $p \leq 10^6$; 大素数不能因查询下标小就忽略这里整段 $O(p)$ 的预处理。fac、ifac 下标为 $0..p-1$, 模数和表只读; 中间乘法用 long long。合数模数请使用 exLucas。

```

1 struct Lucas
2 {
3     int mod;
4     vector<int> fac, ifac;
5
6     Lucas(int p)
7     {
8         assert(p >= 2);
9         mod = p;
10        fac.resize(p);
11        ifac.resize(p);
12        fac[0] = 1;
13        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
14        long long a = fac[p - 1], r = 1;
15        for (int e = p - 2; e; e >>= 1)
16        {
17            if (e & 1) r = r * a % p;
18            a = a * a % p;
19        }
20        ifac[p - 1] = r;
21        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
22    }
23
24    int choose(unsigned long long n, unsigned long long k) const
25    {
26        if (k > n) return 0;
27        long long answer = 1;
28        while (n || k)
29        {
30            int a = n % mod, b = k % mod;
31            if (b > a) return 0;
32            answer = answer * fac[a] % mod;
33            answer = answer * ifac[b] % mod;
34            answer = answer * ifac[a - b] % mod;
35            n /= mod;
36            k /= mod;
37        }
38        return answer;
39    }
40 };

```

模板题使用：Luogu P3807

题意：多组查询 $C(n+m, n)$ 模 p ，题目保证 p 为素数且不超过 10^5 。每组构造 $\text{Lucas}(p)$ ，choose 的参数为 $n+m$ 、 n ；此题范围内 $n+m$ 不会溢出。

所需模板：Lucas（第 40 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3807>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int t;
6     cin >> t;
7     while (t--)
8     {
9         unsigned long long n, m;
10        int p;
11        cin >> n >> m >> p;
12        Lucas solver(p);
13        cout << solver.choose(n + m, n) << '\n';
14    }
15    return 0;
16 }
```

附录 B 组合数学

B.1 排列组合

B.1.1 阶乘组合数

代码依赖：ModInt (第 38 页, A.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 $0..n$, 要求 prime 为素数且 $0 \leq n < \text{prime}$ 。init(n) 只扩大当前上限, 较小或相同的 n 不改变表; choose(n,k) 返回 $C(n,k)$, permute(n,k) 返回 $n!/(n-k)!$ 。查询前显式预处理到 n , 查询本身不扩表。 $n < 0$, $k < 0$ 或 $k > n$ 返回 0, $C(0,0)=P(0,0)=1$ 。fac、ifac 为可直接读取的阶乘与逆阶乘表, 增长可能使旧引用失效。单次查询 $O(1)$; 从 old 扩到 n 新增 $O(n-\text{old}+\log \text{prime})$ 次域运算, 重分配还可能复制旧表。首次预处理 $O(n+\log \text{prime})$, 空间 $O(n)$ 。超过模数不能使用阶乘除法: 小素数用 Lucas, 小合数用 exLucas。

```
1 template <int mod> struct Binomial
2 {
3     using Z = ModInt<mod>;
4     vector<Z> fac{Z(1)}, ifac{Z(1)};
5
6     Binomial(int n = 0) { init(n); }
7
8     void init(int n)
9     {
10         assert(0 <= n && n < mod);
11         int old = (int)fac.size() - 1;
12         if (n <= old) return;
13         fac.resize(n + 1);
14         ifac.resize(n + 1);
15         for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
16         ifac[n] = fac[n].inv();
17         for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
18     }
```

```
20 Z choose(int n, int k) const
21 {
22     if (k < 0 || k > n) return 0;
23     assert(n < (int)fac.size());
24     return fac[n] * ifac[k] * ifac[n - k];
25 }
26
27 Z permute(int n, int k) const
28 {
29     if (k < 0 || k > n) return 0;
30     assert(n < (int)fac.size());
31     return fac[n] * ifac[n - k];
32 }
33 };
```

应用题补充：Luogu P1313 计算系数

题意：求 $(by+ax)$ 的 k 次方中指定项的系数，模 10007。 x 、 y 的指数分别为 n 、 m ，均非负且 $n+m=k$ 。 k 至多 1000， a 、 b 均为不超过 1000000 的非负数。组合 $C(k,n)$ 与两个幂即可。模数为素数且 k 小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：[ModInt](#)（第 38 页，A.1.1 节）、[Binomial](#)（第 42 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P1313>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int a, b, k, n, m;
6     cin >> a >> b >> k >> n >> m;
7     Binomial<10007> c(k);
8     auto x = ModInt<10007>(a).pow(n);
9     auto y = ModInt<10007>(b).pow(m);
10    cout << (c.choose(k, n) * x * y).v << '\n';
11 }
```

附录 C 树状数组

C.1 树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b; b >>= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };
```


模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based, k th 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。 $n \leq 100000$, $|x| \leq 10000000$ ；总复杂度 $O(n \log n)$ ，空间 $O(n)$ 。

所需模板：Fenwick（第 44 页，C.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3369>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     if (!(cin >> n)) return 0;
7     vector<pair<int, int>> q(n);
8     vector<int> a;
9     for (auto &[op, x] : q)
10    {
11        cin >> op >> x;
12        if (op != 4) a.push_back(x);
13    }
14    sort(a.begin(), a.end());
15    a.erase(unique(a.begin(), a.end()), a.end());
16    Fenwick<int> f(a.size());
17    for (auto [op, x] : q)
18    {
19        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
20        if (op == 1)
21            f.add(p + 1, 1);
22        else if (op == 2)
23        {
24            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
25        }
26        else if (op == 3)
27            cout << f.sum(p) + 1 << '\n';
28        else if (op == 4)
29            cout << a[f.kth(x) - 1] << '\n';
30        else if (op == 5)
31            cout << a[f.kth(f.sum(p)) - 1] << '\n';
32        else
33        {
34            p = upper_bound(a.begin(), a.end(), x) - a.begin();
35            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
36        }
37    }
38 }
```

附录 D 线段树

D.1 线段树基础

D.1.1 通用懒标记线段树与边界搜索

模板参数依次为 S、F、op、e、mapping、composition、id；以数组 a 构造，0-based 半开区间；n 构造将 n 个元素置为 e。 $0 \leq n \leq 2^{28}-1$ ，受内存限制。op 满足结合律、e 为双侧单位元；单位标记不改变节点，mapping 必须保持 op 的合并结果；composition(f,g) 表示先 g 后 f，须满足作用复合一致性，id 为标记单位元。无需比较标记是否等于 id，每次下传都应用标记。set/get 单点，apply(l,r,f) 区间作用，prod(l,r) 保序合并，all() 读取根；空区间不更新且查询返回 e。区间长度若参与修改，必须存入 S，不能从 e 推断长度：区间和节点为 {sum,len}，真实叶 len=1，e={0,0}。max_right/min_left 的谓词须满足 ok(e())、无副作用、扩展时真变假后不再变真；两者查询会下传标记，非 const。递归深度 $O(\log n)$ ，构造 $O(n)$ 、操作/搜索 $O(\log n)$ 、all $O(1)$ ，空间 $O(n)$ ，假设运算和类型复制 $O(1)$ 。d/tag 及其他递归辅助方法仅供内部使用。

```
1 template <class S, class F, auto op, auto e, auto mapping, auto composition, auto id>
2 struct lazy_segtree
3 {
4     int n;
5     vector<S> d;
6     vector<F> tag;
7
8     lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n, e())) {}
9
10    lazy_segtree(const vector<S> &a)
11        : n(a.size()), d(4 * n + 4, e()), tag(4 * n + 4, id())
12    {
13        if (n) build(1, 0, n, a);
14    }
15
16    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }
17
18    void build(int p, int l, int r, const vector<S> &a)
19    {
20        if (r - l == 1)
21        {
22            d[p] = a[l];
23            return;
24        }
25        int m = (l + r) / 2;
26        build(p * 2, l, m, a);
27        build(p * 2 + 1, m, r, a);
28        pull(p);
29    }
```

```

31 void act(int p, F f)
32 {
33     d[p] = mapping(f, d[p]);
34     tag[p] = composition(f, tag[p]);
35 }
36
37 void push(int p)
38 {
39     act(p * 2, tag[p]);
40     act(p * 2 + 1, tag[p]);
41     tag[p] = id();
42 }
43
44 void set(int x, S v)
45 {
46     assert(0 <= x && x < n);
47     put(1, 0, n, x, v);
48 }
49
50 void put(int p, int l, int r, int x, S v)
51 {
52     if (r - l == 1)
53     {
54         d[p] = v;
55         tag[p] = id();
56         return;
57     }
58     push(p);
59     int m = (l + r) / 2;
60     if (x < m)
61         put(p * 2, l, m, x, v);
62     else
63         put(p * 2 + 1, m, r, x, v);
64     pull(p);
65 }
66
67 void apply(int l, int r, F f)
68 {
69     assert(0 <= l && l <= r && r <= n);
70     if (l < r) update(1, 0, n, l, r, f);
71 }
72
73 void update(int p, int l, int r, int a, int b, F f)
74 {
75     if (a <= l && r <= b)
76     {
77         act(p, f);
78         return;
79     }
80     push(p);
81     int m = (l + r) / 2;
82     if (a < m) update(p * 2, l, m, a, b, f);
83     if (m < b) update(p * 2 + 1, m, r, a, b, f);
84     pull(p);
85 }

```

```

87     S all() const { return d[1]; }
88
89     S get(int x)
90     {
91         assert(0 <= x && x < n);
92         return prod(x, x + 1);
93     }
94
95     S prod(int l, int r)
96     {
97         assert(0 <= l && l <= r && r <= n);
98         return l == r ? e() : query(1, 0, n, l, r);
99     }
100
101     S query(int p, int l, int r, int a, int b)
102     {
103         if (a <= l && r <= b) return d[p];
104         push(p);
105         int m = (l + r) / 2;
106         if (b <= m) return query(p * 2, l, m, a, b);
107         if (m <= a) return query(p * 2 + 1, m, r, a, b);
108         return op(query(p * 2, l, m, a, b), query(p * 2 + 1, m, r, a, b));
109     }
110
111     template <class G> int max_right(int l, G ok)
112     {
113         assert(0 <= l && l <= n && ok(e()));
114         S s = e();
115         return l == n ? n : right(1, 0, n, l, s, ok);
116     }
117
118     template <class G> int right(int p, int l, int r, int a, S &s, G &ok)
119     {
120         if (r <= a) return n;
121         if (a <= l && ok(op(s, d[p])))
122         {
123             s = op(s, d[p]);
124             return n;
125         }
126         if (r - l == 1) return l;
127         push(p);
128         int m = (l + r) / 2;
129         int x = right(p * 2, l, m, a, s, ok);
130         return x != n ? x : right(p * 2 + 1, m, r, a, s, ok);
131     }

```

```
133     template <class G> int min_left(int r, G ok)
134     {
135         assert(0 <= r && r <= n && ok(e()));
136         S s = e();
137         return r == 0 ? 0 : left(1, 0, n, r, s, ok);
138     }
139
140     template <class G> int left(int p, int l, int r, int b, S &s, G &ok)
141     {
142         if (b <= 1) return 0;
143         if (r <= b && ok(op(d[p], s)))
144         {
145             s = op(d[p], s);
146             return 0;
147         }
148         if (r - l == 1) return r;
149         push(p);
150         int m = (l + r) / 2;
151         int x = left(p * 2 + 1, m, r, b, s, ok);
152         return x != 0 ? x : left(p * 2, l, m, b, s, ok);
153     }
154 };
```

模板题使用：Library Checker range_affine_range_sum

题意：对数组区间施加 $x \rightarrow b * x + c$ ，并查询区间和。S 保存和与长度；空单位元长度 0，单个元素长度 1。composition(f,g) 表示先 g 后 f，mapping 按长度累加常数项。区间为 0-based 半开，模 998244353。

所需模板：lazy_segtree (第 46 页, D.1.1 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_affine_range_sum

```
1 using S = pair<long long, int>;
2 using F = pair<long long, long long>;
3 const int mod = 998244353;
4
5 S op(S a, S b)
6 {
7     return {(a.first + b.first) % mod, a.second + b.second};
8 }
9
10 S e()
11 {
12     return {0, 0};
13 }
14
15 S mapping(F f, S a)
16 {
17     return {(f.first * a.first + f.second * a.second) % mod, a.second};
18 }
19
20 F composition(F f, F g)
21 {
22     return {f.first * g.first % mod, (f.first * g.second + f.second) % mod};
23 }
24
25 F id()
26 {
27     return {1, 0};
28 }
```

使用入口（接上页配置函数）：

```

30 int main()
31 {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34     int n, q;
35     cin >> n >> q;
36     vector<S> a(n);
37     for (auto &[x, len] : a)
38     {
39         cin >> x;
40         len = 1;
41     }
42     lazy_segtree<S, F, op, e, mapping, composition, id> s(a);
43     while (q--)
44     {
45         int t, l, r;
46         cin >> t >> l >> r;
47         if (t == 0)
48         {
49             long long b, c;
50             cin >> b >> c;
51             s.apply(l, r, {b, c});
52         }
53         else
54             cout << s.prod(l, r).first << '\n';
55     }
56 }

```

D.1.2 通用线段树与左右边界搜索

`segtree<S,op,e>(a)` 从 0-based vector 构建，或 `segtree(n)` 初始化为单位元。 $0 \leq n \leq 2^{29}$ ，实际受内存限制。`op(S,S)` 须满足结合律，`e()` 是双侧单位元；无需交换律。`set(p,x)`、`get(p)` 操作单点，`prod(l,r)` 返回半开区间 $[l,r)$ 从左到右的合并，空区间返回 `e`；`all()` 返回全区间结果。`max_right(l,ok)` 返回最大的 r 使 `ok(prod(l,r))` 为真；`min_left(r,ok)` 返回最小的 l 。要求 `ok(e())` 为真，`ok` 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。 $l=n$ 、 $r=0$ 支持空端点。构建 $O(n)$ ，单点/区间/边界搜索 $O(\log n)$ ，`all` $O(1)$ ，空间 $O(n)$ ，假设 `op`、`e`、`ok` 及 `S` 复制为 $O(1)$ 。`d`、`sz`、`pull` 为内部实现，不直接修改。函数复合时 `op(f,g)` 表示 `g(f(x))`，对应先执行左区间再执行右区间；`min_left` 不能照抄 `max_right` 的合并顺序。

```

1  template <class S, auto op, auto e> struct segtree
2  {
3      int n, sz = 1;
4      vector<S> d;
5
6      segtree(int n = 0) : segtree(vector<S>(n, e())) {}
7
8      segtree(const vector<S> &a) : n(a.size())
9      {
10         while (sz < n) sz <= 1;
11         d.assign(2 * sz, e());
12         copy(a.begin(), a.end(), d.begin() + sz);
13         for (int i = sz - 1; i; i--) pull(i);
14     }
15 }

```

```
16 void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }
17
18 void set(int p, S x)
19 {
20     assert(0 <= p && p < n);
21     d[p += sz] = x;
22     while (p >>= 1) pull(p);
23 }
24
25 S get(int p) const
26 {
27     assert(0 <= p && p < n);
28     return d[p + sz];
29 }
30
31 S all() const { return d[1]; }
```



```

33  S prod(int l, int r) const
34  {
35      assert(0 <= l && l <= r && r <= n);
36      S a = e(), b = e();
37      for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
38      {
39          if (l & 1) a = op(a, d[l++]);
40          if (r & 1) b = op(d[--r], b);
41      }
42      return op(a, b);
43  }
44
45  template <class F> int max_right(int l, F ok) const
46  {
47      assert(0 <= l && l <= n && ok(e()));
48      if (l == n) return n;
49      S a = e();
50      l += sz;
51      do
52      {
53          while (!(l & 1)) l >>= 1;
54          if (!ok(op(a, d[l])))
55          {
56              while (l < sz)
57              {
58                  l *= 2;
59                  if (ok(op(a, d[l]))) a = op(a, d[l++]);
60              }
61              return l - sz;
62          }
63          a = op(a, d[l++]);
64      } while ((l & -1) != 1);
65      return n;
66  }

```

```

68  template <class F> int min_left(int r, F ok) const
69  {
70      assert(0 <= r && r <= n && ok(e()));
71      if (r == 0) return 0;
72      S a = e();
73      r += sz;
74      do
75      {
76          --r;
77          while (r > 1 && (r & 1)) r >>= 1;
78          if (!ok(op(d[r], a)))
79          {
80              while (r < sz)
81              {
82                  r = r * 2 + 1;
83                  if (ok(op(d[r], a))) a = op(d[r--], a);
84              }
85              return r + 1 - sz;
86          }
87          a = op(d[r], a);
88      } while ((r & -r) != r);
89      return 0;
90  }
91 };

```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。 $op(f,g)$ 表示先 f 后 g ，单位元为 x ； $prod(l,r)$ 使用 0-based 半开区间，空区间返回单位元。配置函数须和 `main` 一起抄写。

所需模板：`segtree`（第 51 页，D.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```

1 using S = pair<long long, long long>;
2 const int mod = 998244353;
3
4 S op(S f, S g)
5 {
6     return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
7 }
8
9 S e()
10 {
11     return {1, 0};
12 }
13
14 int main()
15 {
16     ios::sync_with_stdio(false);
17     cin.tie(nullptr);
18     int n, q;
19     cin >> n >> q;
20     vector<S> a(n);
21     for (auto &[x, y] : a) cin >> x >> y;
22     segtree<S, op, e> s(a);
23     while (q--)
24     {
25         int t;
26         cin >> t;
27         if (t == 0)
28         {
29             int p;
30             long long c, d;
31             cin >> p >> c >> d;
32             s.set(p, {c, d});
33         }
34         else
35         {
36             int l, r;
37             long long x;
38             cin >> l >> r >> x;
39             auto [c, d] = s.prod(l, r);
40             cout << (c * x + d) % mod << '\n';
41         }
42     }
43 }
```

D.1.3 线段树：区间乘、加、赋值与模意义求和

`AffineSegTree(a,mod)` 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。`update(l,r,mul,add)` 将区间元素 x 改为 $mul \cdot x + add$ ；乘法用 `(mul,0)`，加法用 `(1,add)`，赋值用 `(0,value)`。`query(l,r)` 返回区间和。 $1 \leq mod \leq INT_MAX$ ，模数无需为素数，初值和修改参数允许 long long 全范围，入口统一归一化。旧标记 `(m,b)` 后应用新标记 `(a,c)`，复合为 $(m \cdot a, b \cdot a + c)$ ，乘法取模后再相加。建树 $O(n)$ ，单次 $O(\log n)$ ，空间 $O(n)$ 。内部 `build`、`apply` 等函数不作为公共接口；动态版重建新对象。

```

1 struct AffineSegTree
2 {
3     using ll = long long;
4
5     struct Node
6     {
7         ll sum = 0, mul = 1, add = 0;
8     };
9
10    int n, mod;
11    vector<Node> t;
12
13    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
14    {
15        assert(n > 0 && mod > 0);
16        build(1, 1, n, a);
17    }
18
19    ll norm(ll x) const
20    {
21        x %= mod;
22        if (x < 0) x += mod;
23        return x;
24    }
25
26    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }
27
28    void build(int p, int l, int r, const vector<ll> &a)
29    {
30        t[p] = Node{};
31        if (l == r)
32        {
33            t[p].sum = norm(a[l - 1]);
34            return;
35        }
36        int m = (l + r) / 2;
37        build(p * 2, l, m, a);
38        build(p * 2 + 1, m + 1, r, a);
39        pull(p);
40    }
41
42    // New transform follows the old pending transform: x -> x*a+b.
43    void apply(int p, int len, ll a, ll b)
44    {
45        t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
46        t[p].mul = t[p].mul * a % mod;
47        t[p].add = (t[p].add * a % mod + b) % mod;
48    }

```

```

49
50 void push(int p, int l, int r)
51 {
52     if (t[p].mul == 1 && t[p].add == 0) return;
53     int m = (l + r) / 2;
54     apply(p * 2, m - l + 1, t[p].mul, t[p].add);
55     apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
56     t[p].mul = 1;
57     t[p].add = 0;
58 }
59
60 void update(int l, int r, ll a, ll b)
61 {
62     assert(1 <= l && l <= r && r <= n);
63     update(1, 1, n, l, r, norm(a), norm(b));
64 }
65
66 void update(int p, int l, int r, int ql, int qr, ll a, ll b)
67 {
68     if (ql <= l && r <= qr)
69     {
70         apply(p, r - l + 1, a, b);
71         return;
72     }
73     push(p, l, r);
74     int m = (l + r) / 2;
75     if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
76     if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
77     pull(p);
78 }
79
80 ll query(int l, int r)
81 {
82     assert(1 <= l && l <= r && r <= n);
83     return query(1, 1, n, l, r);
84 }
85
86 ll query(int p, int l, int r, int ql, int qr)
87 {
88     if (ql <= l && r <= qr) return t[p].sum;
89     push(p, l, r);
90     int m = (l + r) / 2;
91     ll answer = 0;
92     if (ql <= m) answer += query(p * 2, l, m, ql, qr);
93     if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
94     return answer % mod;
95 }
96 };

```

模板题使用：Luogu P3373

题意：对数组执行区间乘、区间加和区间求和，结果对给定模数取模。区间为 1-based 闭区间；update 的两个参数表示先乘再加。

所需模板：AffineSegTree (第 56 页, D.1.3 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3373>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     int n, q, mod;
6     std::cin >> n >> q >> mod;
7     std::vector<long long> a(n);
8     for (auto &x : a) std::cin >> x;
9     AffineSegTree tree(a, mod);
10    while (q--)
11    {
12        int op, l, r;
13        std::cin >> op >> l >> r;
14        if (op == 3)
15            std::cout << tree.query(l, r) << '\n';
16        else
17        {
18            long long k;
19            std::cin >> k;
20            if (op == 1)
21                tree.update(l, r, k, 0);
22            else
23                tree.update(l, r, 1, k);
24        }
25    }
26 }
```

附录 E 树上问题

E.1 树链剖分

E.1.1 重链剖分与 LCA

`add(u,v)` 建无向树; `build(root)` 后读取 `fa`、`dep`、`siz`、`son`、`top`、`dfn`、`rk`。子树对应 $[dfn[u], dfn[u] + siz[u] - 1]$ 。`lca(u,v)` 为 $O(\log n)$, `path(u,v,work,edge)` 分解路径; `work(l,r)` 仅适用于可交换聚合, 边权模式排除 LCA 位置。输入必须是连通无环树, 建树 $O(n)$, 两遍递归 DFS; 支持改变根后重复 `build`, `dfn` 会重新编号。`path_ordered(u,v,work,edge)` 的 `edge` 默认为 `false`; 按真实 `u` 到 `v` 顺序回调 `work(l,r,rev)`: 1-based 闭区间 $l \leq r$, `rev=true` 按 `r` 到 `l` 访问, 否则按 `l` 到 `r`。支持非交换聚合, $O(\log n)$ 段与额外空间; 结合线段树通常每次查询 $O(\log^2 n)$ 。`edge=true` 仅删除 LCA 对应位置, 仍须自行处理边算子的方向; 不自动求逆。`u=v` 时点模式一次单点回调, 边模式无回调; 改变根后须重建依赖 `dfn` 的结构。

```
1 struct HLD
2 {
3     int n, timer = 0;
4     vector<vector<int>> g;
5     vector<int> fa, dep, siz, son, top, dfn, rk;
6
7     HLD(int n)
8         : n(n),
9           g(n + 1),
10          fa(n + 1),
11          dep(n + 1),
12          siz(n + 1),
13          son(n + 1),
14          top(n + 1),
15          dfn(n + 1),
16          rk(n + 1)
17     {
18     }
19
20     void add(int u, int v)
21     {
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     void dfs1(int u, int p)
27     {
28         fa[u] = p;
29         siz[u] = 1;
30         son[u] = 0;
```

```
31     for (int v : g[u])
32     {
33         if (v == p) continue;
34         dep[v] = dep[u] + 1;
35         dfs1(v, u);
36         siz[u] += siz[v];
37         if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
38     }
39 }
```


重编号、LCA 与可交换路径（接上页同一结构体）：

```

41 void dfs2(int u, int t)
42 {
43     top[u] = t;
44     dfn[u] = ++timer;
45     rk[timer] = u;
46     if (son[u]) dfs2(son[u], t);
47     for (int v : g[u])
48     {
49         if (v != fa[u] && v != son[u]) dfs2(v, v);
50     }
51 }
52
53 // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
54 void build(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     timer = 0;
58     dep[root] = 0;
59     dfs1(root, root);
60     dfs2(root, root);
61     assert(timer == n);
62 }
63
64 int lca(int u, int v) const
65 {
66     while (top[u] != top[v])
67     {
68         if (dep[top[u]] < dep[top[v]]) swap(u, v);
69         u = fa[top[u]];
70     }
71     return dep[u] < dep[v] ? u : v;
72 }
73
74 // Commutative operations only; edge=true excludes the LCA's position.
75 template <class F> void path(int u, int v, F work, bool edge = false) const
76 {
77     while (top[u] != top[v])
78     {
79         if (dep[top[u]] < dep[top[v]]) swap(u, v);
80         work(dfn[top[u]], dfn[u]);
81         u = fa[top[u]];
82     }
83     if (dep[u] > dep[v]) swap(u, v);
84     if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
85 }

```

按真实方向分解路径（接上页同一结构体）：

```
87 // Emit u-to-v order; reverse=true traverses the closed interval from r to l.
88 template <class F> void path_ordered(int u, int v, F work, bool edge = false) const
89 {
90     vector<pair<int, int>> down;
91     while (top[u] != top[v])
92     {
93         if (dep[top[u]] >= dep[top[v]])
94         {
95             work(dfn[top[u]], dfn[u], true);
96             u = fa[top[u]];
97         }
98         else
99         {
100             down.emplace_back(dfn[top[v]], dfn[v]);
101             v = fa[top[v]];
102         }
103     }
104     if (dep[u] >= dep[v])
105     {
106         if (dfn[v] + edge <= dfn[u]) work(dfn[v] + edge, dfn[u], true);
107     }
108     else if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v], false);
109     for (auto it = down.rbegin(); it != down.rend(); ++it)
110         work(it->first, it->second, false);
111 }
112 };
```

模板题使用：Luogu P3384

题意：给定有根点权树，支持路径加、路径和、子树加、子树和，答案模 mod 。先按 dfn 重排初值再构造线段树； path 回调给出 1-based 闭区间，子树区间为 $[\text{dfn}[u], \text{dfn}[u] + \text{siz}[u] - 1]$ 。

所需模板：[HLD](#)（第 59 页，E.1.1 节）、[AffineSegTree](#)（第 56 页，D.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3384>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, root, mod;
6     cin >> n >> m >> root >> mod;
7     vector<long long> values(n + 1);
8     for (int u = 1; u <= n; u++) cin >> values[u];
9     HLD h(n);
10    for (int i = 1; i < n; i++)
11    {
12        int u, v;
13        cin >> u >> v;
14        h.add(u, v);
15    }
16    h.build(root);
17    vector<long long> ordered(n);
18    for (int u = 1; u <= n; u++) ordered[h.dfn[u] - 1] = values[u];
19    AffineSegTree seg(ordered, mod);
20    while (m--)
21    {
22        int op, u;
23        cin >> op >> u;
24        if (op == 1)
25        {
26            int v;
27            long long x;
28            cin >> v >> x;
29            h.path(u, v, [&](int l, int r) { seg.update(l, r, 1, x); });
30        }
31        else if (op == 2)
32        {
33            int v;
34            cin >> v;
35            long long ans = 0;
36            h.path(u, v, [&](int l, int r) { ans = (ans + seg.query(l, r)) % mod; });
37            cout << ans << '\n';
38        }
39        else if (op == 3)
40        {
41            long long x;
42            cin >> x;
43            seg.update(h.dfn[u], h.dfn[u] + h.siz[u] - 1, 1, x);
44        }
45        else
46            cout << seg.query(h.dfn[u], h.dfn[u] + h.siz[u] - 1) << '\n';
47    }
48 }

```

模板题使用：Library Checker Vertex Set Path Composite

题意：静态无向树， $1 \leq N, Q \leq 200000$ 。点函数 $f(x) = a * x + b$ ，对 998244353 取模；题面 $1 \leq a, c < \text{mod}, 0 \leq b, d, x < \text{mod}$ 。
 $0 \leq p < d$ 改单点函数， $1 \leq u, v \leq x$ 按 u 到 v （含两端点）依次代入。题目点号 0-based，传 HLD 时加 1；HLD 闭区间 $[l, r]$ 对应 segtree 半开 $[l-1, r)$ 。S 存正向与反向复合，不能把组合顺序反过来或用逆函数代替反向序列。点改 $O(\log N)$ ，路径 $O(\log^2 N)$ ，空间 $O(N)$ ；保留递归 DFS，最大链须足够大栈。

所需模板：HLD（第 59 页，E.1.1 节）、segtree（第 51 页，D.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/vertex_set_path_composite

```

1 using S = array<long long, 4>;
2 const int mod = 998244353;
3
4 S op(S a, S b)
5 {
6     return {b[0] * a[0] % mod,
7             (b[0] * a[1] + b[1]) % mod,
8             a[2] * b[2] % mod,
9             (a[2] * b[3] + a[3]) % mod};
10 }
11
12 S e()
13 {
14     return {1, 0, 1, 0};
15 }
16
17 int main()
18 {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21     int n, q;
22     cin >> n >> q;
23     vector<S> a(n), val(n);
24     for (auto &x : a)
25     {
26         cin >> x[0] >> x[1];
27         x[2] = x[0];
28         x[3] = x[1];
29     }
30     HLD h(n);
31     for (int i = 1; i < n; i++)
32     {
33         int u, v;
34         cin >> u >> v;
35         h.add(u + 1, v + 1);
36     }
37     h.build();
38     for (int u = 1; u <= n; u++) val[h.dfn[u] - 1] = a[u - 1];
39     segtree<S, op, e> s(val);

```

继续处理操作（接上页同一 main）：

```
40     while (q--)  
41     {  
42         int t;  
43         cin >> t;  
44         if (t == 0)  
45         {  
46             int p;  
47             long long c, d;  
48             cin >> p >> c >> d;  
49             s.set(h.dfn[p + 1] - 1, {c, d, c, d});  
50         }  
51         else  
52         {  
53             int u, v;  
54             long long x;  
55             cin >> u >> v >> x;  
56             h.path_ordered(u + 1, v + 1, [&](int l, int r, bool rev)  
57             {  
58                 S z = s.prod(l - 1, r);  
59                 int k = rev ? 2 : 0;  
60                 x = (z[k] * x + z[k + 1]) % mod;  
61             });  
62             cout << x << '\n';  
63         }  
64     }  
65 }
```

接口用法演示：路径点权增减与点查 (kuangbin 3.3.1 源码模型)

演示约定：读到 EOF：n,m,q、n 个初值、m 条树边，要求 $m=n-1$ 。I u v x/D u v x 给路径所有点加/减 x，Q u 查点值。1-based，初值保留在 a，Fenwick 仅记录增量差分；HLD 闭区间 [l,r] 在 l 加 x、r+1 减 x，包含 LCA 与两端。演示 $n,q \leq 50000$ 、| 初值 $\leq 10^{12}$ 、 $0 \leq x \leq 10^6$ ，全部中间值用 long long。每次更新 $O(\log^2 n)$ 、点查 $O(\log n)$ 、空间 $O(n)$ 。原 HDU3966 题面未核验。

所需模板：HLD（第 59 页，E.1.1 节）、Fenwick（第 44 页，C.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://github.com/kuangbin/ACM-ICPC>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m, q;
6     while (cin >> n >> m >> q)
7     {
8         vector<long long> a(n + 1);
9         for (int u = 1; u <= n; u++) cin >> a[u];
10        HLD h(n);
11        for (int i = 0; i < m; i++)
12        {
13            int u, v;
14            cin >> u >> v;
15            h.add(u, v);
16        }
17        h.build();
18        auto bit = Fenwick<long long>(n);
19        while (q--)
20        {
21            char op;
22            int u;
23            cin >> op >> u;
24            if (op == 'Q') cout << a[u] + bit.sum(h.dfn[u]) << '\n';
25            else
26            {
27                int v;
28                long long x;
29                cin >> v >> x;
30                if (op == 'D') x = -x;
31                h.path(u, v, [&](int l, int r)
32                {
33                    bit.add(l, x);
34                    bit.add(r + 1, -x);
35                });
36            }
37        }
38    }
39 }
```

应用题补充：SPOJ QTREE / 单边修改与路径最大边权

题意：多测 $t \leq 20$ 、 $n \leq 10000$ 。输入边按 $1..n-1$ 编号；CHANGE $i\ x$ 修改第 i 条边，QUERY $u\ v$ 查路径最大边权，DONE 结束本组。把边存到较深端点的 $dfn-1$ ；path 的 $edge=true$ 排除 LCA，闭 $[l,r]$ 转为 segtree 的半开 $[l-1,r)$ 。空路径约定输出 0，非空路径支持 signed64 负权及最小值；题面仅写 $c \leq 10^6$ ，未写修改值下界。预处理 $O(n)$ 、修改 $O(\log n)$ 、查询 $O(\log^2 n)$ 、空间 $O(n)$ ，递归 DFS。

所需模板：HLD（第 59 页，E.1.1 节）、segtree（第 51 页，D.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.spoj.com/problems/QTREE/>

```
1 auto op(long long x, long long y)
2 {
3     return max(x, y);
4 }
5
6 auto e()
7 {
8     return numeric_limits<long long>::lowest();
9 }
```

使用入口（接上页配置函数）：

```

11 int main()
12 {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15     int tests;
16     cin >> tests;
17     while (tests--)
18     {
19         int n;
20         cin >> n;
21         HLD h(n);
22         vector<int> a(n), b(n), child(n);
23         vector<long long> w(n), base(n, e());
24         for (int i = 1; i < n; i++)
25         {
26             cin >> a[i] >> b[i] >> w[i];
27             h.add(a[i], b[i]);
28         }
29         h.build();
30         for (int i = 1; i < n; i++)
31         {
32             child[i] = h.dep[a[i]] > h.dep[b[i]] ? a[i] : b[i];
33             base[h.dfn[child[i]] - 1] = w[i];
34         }
35         auto seg = segtree<long long, op, e>(base);
36         string cmd;
37         while (cin >> cmd && cmd != "DONE")
38         {
39             int u;
40             cin >> u;
41             if (cmd == "CHANGE")
42             {
43                 long long x;
44                 cin >> x;
45                 seg.set(h.dfn[child[u]] - 1, x);
46             }
47             else
48             {
49                 int v;
50                 cin >> v;
51                 long long ans = e();
52                 h.path(u, v, [&](int l, int r)
53                 {
54                     ans = max(ans, seg.prod(l - 1, r));
55                 }, true);
56                 cout << (u == v ? 0 : ans) << '\n';
57             }
58         }
59     }
60 }

```


接口用法演示：树边翻转与奇路径计数（kuangbin HDU5039 模型）

演示约定：来源协议：T 组，每组 n 、 n 个不同名字、 $n-1$ 条（名字，名字，0/1）边、 q 条操作；Q 开头查询，其余操作接 1-based 输入边号并翻转该边。输出 Case #k: 及每次有向端点对数量，故答案 $2^{\text{zero} \cdot \text{one}}$ ；无向点对则不乘 2。固定根 1，根路径异或值不同的端点间为奇路径；翻边即翻较深端点整棵子树，HLD 闭区间转 lazy_segtree 半开 $[\text{dfn}-1, \text{dfn}-1+\text{siz})$ 。树上预处理 $O(n)$ ，名字 map 映射另计字符串比较和字典查询；改边 $O(\log n)$ ，查询 $O(1)$ ，空间 $O(n)$ ，DFS 递归。官方 HDU 题面和限制未核验，只作来源模型/API 用法。

所需模板：HLD（第 59 页，E.1.1 节）、lazy_segtree（第 46 页，D.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://acm.hdu.edu.cn/showproblem.php?pid=5039>

```

1 using S = pair<int, int>;
2
3 S op(S a, S b)
4 {
5     return {a.first + b.first, a.second + b.second};
6 }
7
8 S e()
9 {
10    return {0, 0};
11 }
12
13 S mapping(bool flip, S a)
14 {
15     if (flip) swap(a.first, a.second);
16     return a;
17 }
18
19 bool composition(bool f, bool g)
20 {
21     return f ^ g;
22 }
23
24 bool id()
25 {
26     return false;
27 }
```

使用入口（接上页配置函数）：

```

29 int main()
30 {
31     ios::sync_with_stdio(false);
32     cin.tie(nullptr);
33     int tests;
34     cin >> tests;
35     for (int tc = 1; tc <= tests; tc++)
36     {
37         int n;
38         cin >> n;
39         HLD h(n);
40         map<string, int> names;
41         for (int u = 1; u <= n; u++)
42         {
43             string name;
44             cin >> name;
45             names[name] = u;
46         }
47         vector<int> u(n), v(n), bit(n), child(n), parity(n + 1);
48         for (int i = 1; i < n; i++)
49         {
50             string a, b;
51             cin >> a >> b >> bit[i];
52             u[i] = names.at(a);
53             v[i] = names.at(b);
54             h.add(u[i], v[i]);
55         }
56         h.build();
57         for (int i = 1; i < n; i++)
58         {
59             child[i] = h.fa[u[i]] == v[i] ? u[i] : v[i];
60             parity[child[i]] = bit[i];
61         }
62         vector<S> values(n);
63         for (int i = 1; i <= n; i++)
64         {
65             int x = h.rk[i];
66             if (x != 1) parity[x] ^= parity[h.fa[x]];
67             values[i - 1] = {parity[x] == 0, parity[x] == 1};
68         }
69         lazy_segtree<S, bool, op, e, mapping, composition, id> seg(values);

```

继续处理操作（接上页同一 main）：

```
70     int q;
71     cin >> q;
72     cout << "Case #" << tc << ":\n";
73     while (q--)
74     {
75         string command;
76         cin >> command;
77         if (command[0] == 'Q')
78         {
79             auto [zero, one] = seg.all();
80             cout << 2LL * zero * one << '\n';
81         }
82         else
83         {
84             int edge;
85             cin >> edge;
86             int x = child[edge];
87             int l = h.dfn[x] - 1;
88             seg.apply(l, l + h.siz[x], true);
89         }
90     }
91 }
92 }
```

算法索引

AffineSegTree, 56

Binomial, 42

Chain3D, 12

Dominance3D, 10

ExactCover, 1

Fenwick, 44

HLD, 59

lazy__segtree, 46

Lucas, 40

MinimumCover, 5

ModifiedMo, 21

ModInt, 38

OverallKth, 17

RollbackMo, 28

segtree, 51

TreeMo, 24

xor__hamming__pairs, 32

带修莫队, 35